

Greedy Algorithms

CSC 349 - Algorithms

Cal Poly, San Luis Obispo

Rodrigo Canaan

Adapted from materials by Alexander Dekthyar, Theresa Miggler

Agenda

- Introduction
 - Change Making
 - Optimization Problems
- Greedy Algorithms in Graphs
 - Dijkstra
 - Minimum Spanning Tree
- Huffman Coding
- Exercises

1 - Introduction

**Introduction - Change Making
Optimization Problems**

Introduction

- A greedy algorithm is one that makes a “locally optimal” choice at each step
 - No lookahead
 - No “undoing” of previous steps

Example

Change-Making

- Suppose you have an infinite supply of coins worth [1, 5, 10, 25, 50, 100] cents
- How would you make change for 237 (in other words, 2 dollars and 37 cents), using as few coins as possible?
- Intuition: we tend to use fewer coins in total if we use high-value coins preferentially
- Greedy approach: take the highest-valued coin, subtract from the target, repeat.

Example

Change-Making

Iteration	Target	Coin to use	New target	Total number of coins
0	237	100	137	1
1	137	100	37	2
2	37	25	12	3
3	12	10	2	4
4	2	1	1	5
5	1	1	0	6

Example

Change-Making

- The greedy solution therefore uses 6 coins: 100, 100, 25, 10, 1, 1
- We can represent this solution as an array [2,0,1,1,0,2] representing the number of times we used the coins worth [1,5,10,25,50,100] respectively

Problem definition - Change Making

Input: an array $A = [a_1, a_2, a_3, \dots, a_k]$ where $a_i < a_{i+1} \forall i$ and a target T . We will assume $a_1 = 1$

Output: an array $Change = [c_1, c_2, c_3, \dots, c_k]$ such that:

a. $A \cdot Change = \sum_{i=1}^k a_i c_i = T$

b. $\sum_{i=1}^k c_i$ (the total number of coins used) is minimal

Example

Change-Making

- Pseudocode:

Greedy Change-Making (A, T)

Input:

Output:

1. Initialize $\text{Change}[i] \leftarrow 0$ for $i = 1, 2, \dots, k$
2. $\text{current} \leftarrow k$
3. While $T > 0$
4. if $T \geq A[\text{current}]$
5. $T \leftarrow T - A[\text{current}]$
6. $\text{Change}[\text{current}] \leftarrow \text{Change}[\text{current}] + 1$
7. else
8. $\text{current} \leftarrow \text{current} - 1$
9. return Change

Optimization problems

Definition

- Greedy algorithms are often used to solve *optimization problems*
 - Informally, min/maximize some quantity, subject to some restrictions
- We will define an optimization problem by:
 - A set of instances (inputs)
 - A set of decision variables (outputs)
 - One or more constraints over the decision variables
 - An objective function over the decision variables
- The goal is to find an assignment of values to the decision variables that:
 - satisfies all the constraints while
 - optimizing (minimizing or maximizing) the objective function

Exercise:

- Identify, for Change-Making
 - What is the set of instances?
 - What are the decision variables?
 - What are the constraints?
 - What is the objective function?

Optimization problems

Definition

- Terminology (decision problem)
 - Any assignment of values to the decision variables is a *candidate solution* or simply a *candidate*
 - An assignment that satisfies all the constraints is called a *feasible solution* or simply a *solution*
 - A feasible solution that also optimizes the objective function is called an *optimal solution*
 - In other words, no other feasible solution results in a smaller (if minimizing) or higher (if maximizing) evaluation by the objective function

Optimization problems

Definition

- Terminology (algorithm properties)
 - We will say that an algorithm is *correct* if it always returns a feasible solution if one exists (and returns FAIL if no feasible solution exists).
 - We will say that an algorithm is *optimal* if it always returns an optimal solution (assuming at least one feasible solution exists)

Optimization problems

Definition

- Claim: greedy change-making is correct
 - Proof: induction in T
 - This assumes that $a_1 = 1$
 - Come up with a counterexample for A, T where the algorithm fails
 - What needs to be changed in that case?

Optimization problems

Definition

- Claim: greedy change-making is optimal if $A = [1, 5, 10, 25, 50, 100]$ (the American coin system)
 - This is harder to prove
 - Come up with a counterexample for A , T where the algorithm finds a solution, but where the solution isn't optimal?
 - A coin system (choice of A) where the greedy algorithm is always optimal is called *canonical*
 - See <https://math.stackexchange.com/questions/3121896/what-property-of-a-coin-system-makes-it-canonical> (access Feb 13, 2025)

Exercises

1. Modify the given pseudocode such that it correctly returns FAIL if there is no feasible solution for a given choice of A, T
 - (for example, if $A = [2, 5, 10], T = 3$)
2. Come up with a choice of A, T for which the algorithm returns a feasible, but not optimal solution

**A Greedy Algorithm
isn't necessarily
optimal!!!!**

Why greedy?

- Greedy algorithms aren't always optimal (like for some instances of change-making)
- For some interesting problems, however, they are
- They are also usually very fast to come up with approximate (non-optimal) solutions